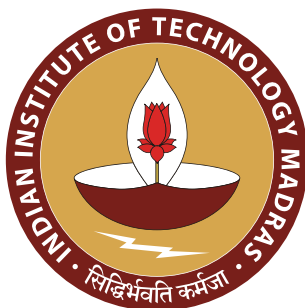


Learning from Demonstrations - State-only Imitation Learning

Submitted by: **Adithyaa R G**

Roll no: **ED21B005**



**Interdisciplinary Dual Degree:
Robotics**

Endterm Dual Degree Project Report

September 6, 2026

Report for Endterm Dual Degree Project

Interdisciplinary Dual Degree: Robotics, IIT Madras

1. **Name of the Student** : Adithyaa R G
2. **Registration Number** : ED21B005
3. **Registered for** : B.Tech Engineering Design + M.Tech IDDD Robotics
4. **Name(s) of the Supervisor** : Prof. B Ravindran, Prof. Nirav Bhatt

ADITHYAA R G
Signature of the student

PROF. B RAVINDRAN
Signature of the advisor

PROF. NIRAV BHATT
Signature of the advisor

Acknowledgements

I would like to express my sincere gratitude to my mentor, **Returaj Burnwal**, and my advisors, **Prof. B Ravindran** and **Prof. Nirav Bhatt**, for their constant guidance, insightful discussions, and support throughout this project. Their perspectives and feedback have played a crucial role in shaping both the direction and depth of this work.

I am also thankful to my lab members for the engaging discussions and constructive suggestions, which have significantly contributed to refining the ideas and experiments presented in this report.

I would like to extend my heartfelt gratitude to my parents, brother and friends at IIT Madras for their constant encouragement and support. Their motivation and belief in me have been instrumental throughout my academic journey.

Finally, I would like to acknowledge **IIT Madras** for providing a stimulating academic environment and the resources necessary to carry out this research.

Contents

1	Abstract	6
2	Introduction	6
2.1	Reinforcement Learning	6
2.2	Challenges with Real World Deployment	7
2.3	Imitation Learning	7
2.4	Challenges with existing methods	8
3	Related Work	8
3.1	Learning from State only Demonstrations	8
3.2	Density Estimation	9
3.3	Likelihood Metrics	10
3.4	Positive Unlabeled Learning (PU Learning)	10
4	Methodology	11
4.1	Objective	11
4.2	Approach	12
4.2.1	Sub-Trajectory <i>State-Flow</i> Identification.	12
4.2.2	State Importance Estimation	13
4.2.3	Weighted Imitation	18
5	Experiments and Observations	21
5.1	Normalization in Density Estimation	21
5.2	Behaviour Cloning works only on Hopper	25
5.2.1	Performance on Hopper	25
5.3	Performance on other environments	26
5.3.1	Iterative Training	27
5.3.2	Positive Unlabelled Learning	27
5.3.3	Behaviour Cloning with Original Data	29
6	Points of Failure - Baseline, Distributional Shift in Behaviour Cloning	31
7	Appendix	35
7.1	Algorithm - Iterative Training	35
7.2	Potential Approaches for Parameter Tuning	36
7.3	Baselines	37

List of Figures

1	State Importance Estimation	12
2	The difference in rewards between the true rewards and estimated rewards caused by binary discrimination between expert and non-expert datasets. Ideally, only states in green region can be given rewards with high confidence.	14
3	Rewards for expert and non-expert transitions (1000 trajectories each) for the hopper agent.	15
4	Logarithmic score distribution of generated samples in the Hopper dataset. The values span a wide range (approximately -2200 to 65) and are concentrated in the negative region, which makes direct exponentiation numerically unstable.	21
5	Effect of log-density scaling on sample weighting.	22
6	Effect of normalization and exponentiation on log-density scores.	23
7	Comparison of original reward distributions for halfcheetah for different state-flow sequence lengths. Longer sequences result in improved separation between expert and non-expert trajectories, particularly for the expert trajectories.	24
8	Exponentiated score distributions across different environments. The trends remain consistent, showing controlled scaling and clear separation between expert, medium, and simple trajectory qualities.	24
9	Performance comparison of Behaviour Cloning with baseline methods in the Hopper environment.	25
10	Training performance in the HalfCheetah environment, showing that the learned policy fails to exceed the baseline normalized reward of 0 , corresponding to simple trajectories.	26
11	$\bar{w} = \log P(\bar{s}' \bar{s})$, where $\bar{s}'$ denotes the predicted sequence and $\bar{s}$ the conditioned sequence of states. With PU learning, the density model effectively distinguishes between expert, medium, and simple data after constructing state flows.	28
12	Comparison of reward scoring on hopper dataset samples. X-axis represents the number of samples used to train the density model with PU learning. The X-axis shows the increasing number of training steps with size 1000 steps per index. . . .	28
13	Behaviour Cloning performance under different weighting schemes. Both variants struggle to learn effectively despite return-based weighting.	30
14	Effect of different baseline selection strategies on behaviour cloning performance. In Fig. 14a, lowering the temperature and using a baseline of $\mu + 1\sigma$ leads to occasional high-performing trajectories, with some rollouts achieving normalized scores above 50 . However, this improvement is inconsistent across evaluations. In Fig. 14b, selecting a conservative baseline based on the worst-case expert trajectories produces a similar pattern: sporadic high returns without stable performance. These results suggest that while the policy is capable of discovering high-reward behaviours, it fails to reproduce them reliably. This inconsistency is likely due to distributional shift, where the learned policy deviates from the training distribution and cannot consistently recover favourable trajectories.	30
15	Illustration of multi-modal expert trajectories. Distinct modes in the expert data can lead Behaviour Cloning to interpolate between trajectories, potentially resulting in unrealistic or low-density paths.	32

-
- 16 Comparison of LobsDICE and SModICE across four MuJoCo environments under varying expert dataset sizes and mixture distributions. Curves show mean return over training with shaded standard deviation across seeds. 38

1 Abstract

Deep Offline RL and Imitation Learning(IL) learn agents from data. They are prone to distributional shift, which hinders their learning. They make extreme assumptions about access to rewards and expert actions in the case of IL. In most situations, it is challenging to collect ego-centric data, and even more difficult to collect action data from agents. This sparsity can be addressed by transfer of skills between agents with identical state-space representations. This method assumes access to a few state-only expert trajectories from a demonstrator agent and to large quantities of state-action trajectories from a skilled imitator agent. The imitator agent learns a policy from these through a two-step process: (i) Identifying and scoring state transitions in the imitator dataset based on the expert trajectories from the demonstrator agent. Mixture Models are used to model the conditional density of state transitions in both the demonstrator and imitator datasets. An importance score is calculated based on the difference in the likelihoods for each transition under these density models. (ii) Use Weighted Imitation Learning techniques to learn a policy mapping between the state and the action. The effectiveness of these methods is tested and evaluated on regular offline datasets, with expert demonstrations and mixed imitator datasets.

2 Introduction

2.1 Reinforcement Learning

Reinforcement Learning (RL) Sutton and Barto (1998) provides a framework for learning control policies through interaction with an environment. RL has achieved impressive results in domains such as manipulation, locomotion, and humanoid control. RL learns a policy that maximizes an expected cumulative reward (return), characterized by the *Hamilton-Jacobi-Bellman* (HJB) Yong and Zhou (1999) equations. In theory, dynamic programming would solve the equations, these methods scale poorly with dimensionality, leading to the well-known *curse of dimensionality*.

Deep RL alleviates the scalability issues by parameterizing value and policy functions using neural networks. *Value-based methods* approximate the optimal action-value function, while *policy gradient methods* optimize a differentiable policy directly, both optimizing for the same objective of maximizing the expected returns. *Actor-Critic* Konda and Tsitsiklis (1999) algorithms combine these paradigms by learning both a parameterized policy (actor) and a value function (critic), often using advantage-based updates for stability.

2.2 Challenges with Real World Deployment

Despite algorithmic advances, deploying RL in physical systems is difficult because real-world exploration is expensive and potentially unsafe. Key challenges Dulac-Arnold et al. (2019) include:

- **Sample Inefficiency and Requirement of Virtual Environment:** Modern RL requires millions of interactions, which are impractical on real robots. We have limited access to data and time to interact with the real environment, and there is a need to create more data-efficient algorithms.
- **Safety:** In real-world systems, naive exploration can risk hardware damage or violate safety constraints. Thus, certain states must be strictly avoided. This requires reinforcement learning methods that incorporate safety-critical constraints during exploration and policy execution.
- **Transferability:** Policies trained in simulation often fail to transfer reliably to real-world systems because simulations, despite their mathematical precision, cannot fully capture the complexity and variability of real environments.

These limitations motivate learning methods that reduce or eliminate unsafe exploration and learn from demonstrations rather than from interactions with the environment, thereby avoiding the need for a virtual learning environment.

2.3 Imitation Learning

Imitation Learning (IL) aims to learn policies directly from trajectories data, thereby avoiding the need for explicit reward design and reducing unsafe exploration. Given a dataset of trajectories (generally *state-action* pairs), IL seeks to learn a policy that reproduces the demonstrator’s behaviour.

Behaviour Cloning (BC) Abbeel and Ng (2004) treats policy learning as supervised learning, directly mapping states to expert actions and aims to maximize the likelihood estimate of the action given a state. Although it learns a policy, BC suffers from distribution shift: errors compound as the learned policy encounters states not represented in the demonstrations.

Distribution matching methods addressed this challenge. The new objective is to minimize the divergence between the state transitions between learnt policy and the demonstrator’s dataset. It formed the foundation for a variety of Offline RL methods, including *Adversarial IL* Ho and Ermon (2016) methods.

2.4 Challenges with existing methods

Despite progress, most IL methods assume access to expert datasets with both state and action trajectories. In many realistic settings, only state observations are available. In both ego-centric and non-egocentric datasets, the trajectories may not include any action information. Collecting datasets with action information is also a challenging task, especially in non-egocentric situations.

This project deals with egocentric data points. It considers that state-only expert trajectories exist, and that non-expert trajectories contain information about actions. That would be the first logical relaxation of the existing assumptions.

3 Related Work

3.1 Learning from State only Demonstrations

There have been two primary approaches to State only Demonstration learning. One popular approach is to learn an inverse dynamics models to predict action given a state transitions. However, there are two major caveats to this approach. They assume access to either an underlying simulator or reward information, as seen in Cao and Sadigh (2021), Cao et al. (2021). They also require learning an inverse dynamics model. These suffer from distributional shift and bootstrapping error when the agent’s trajectories diverge from the demonstrations.

The second approach in frameworks such as SMODICE Ma et al. (2022a) and LobsDICE Kim et al. (2022a) address the problem of state-only imitation learning by matching state/state transition distributions instead of state-action pairs. They assume access to a state-only dataset of an expert and a state-action dataset of mixed behaviour from the imitator agent.

$$\mathcal{D}_E = \{s_i\}_{i=1}^{N_E}$$

where N_E is the number of expert trajectories, contains states only, while an imitator dataset,

$$\mathcal{D}_I = \{(s_j, a_j)\}_{j=1}^{N_I}$$

where N_I is the number of imitator trajectories, originates from a suboptimal policy

Let $d_E(s)$ and $d_\pi(s)$ denote the stationary state distributions of the expert and learner. The objective becomes

$$\min_{\pi} D_f(d_\pi(s) \parallel d_E(s)),$$

where D_f is an f -divergence. The Fenchel–Legendre dual of this problem yields

$$\min_{\pi} \max_x \mathbb{E}_{s \sim d_{\pi}}[x(s)] - \mathbb{E}_{s \sim d_E}[f^*(x(s))],$$

where f^* is the convex conjugate of f . The dual variable $x(s)$ acts as a discriminator estimating relative likelihoods. Solving the objective function leads to a Weighted Behaviour Cloning formulation, with a minimization of the objective:

$$\mathcal{L}_{WBC}(\pi) = -\mathbb{E}_{(s,a) \sim \mathcal{D}_I}[w(s) \log \pi(a|s)].$$

Newer works Yan et al. (2023a) build on SMODICE and LobsDICE. They improve the efficiency of these algorithms by modifying the discriminator. While SMODICE treats all data in the imitator dataset as belonging to the same class, this paper addresses this by training a debiased discriminator that actively identifies states in the imitator dataset that are not from the demonstrator.

Sikchi et al. (2024) proposes an alternative approach to the same problem, based on imitation learning using state visitation frequency. The errors propagated by the chosen action are avoided by learning a multi-step utility function that quantifies the action’s impact and by applying the principle of duality to derive an algorithm that can leverage arbitrary suboptimal data to learn imitator policies. Similarly, Yan et al. (2023b) provides promising improvements over existing methods by using Primal Wasserstein Divergence, an alternative distribution-matching technique to KL Divergence that maximises the agent’s performance while satisfying the Lagrangian and Bellman operator constraints.

3.2 Density Estimation

The discriminators in existing DICE methods make assumptions about overlap between the demonstrator and imitator state distributions. However, in reality, that is an extreme assumption that could hinder performance. We can instead approximate the state distribution using probability density estimators such as Kernel Density Estimators. However, these fail at higher dimensions due to high volumes of data points. Levine et al. (2011) uses a Gaussian Process prior over the reward function, giving a Bayesian posterior over rewards and induced behaviours rather than directly fitting a data density. Uncertainty-aware inference for Inverse Reinforcement Learning (IRL) uses Gaussian Process priors. Mandyam, Li, Jones, Cai, and Engelhardt (Mandyam et al.) applies kernel density estimation to model required likelihood terms in Bayesian IRL. It allows a flexible nonparametric density estimate without assuming a fixed parametric form.

3.3 Likelihood Metrics

Building on the theoretical foundations of the paper Ren et al. (2019) Assume that $p_\theta(s'|s)$ is a model trained to predict the next state, given the current state. It functions as our in-distribution model, and $p_{\theta_0}(s')$ is a background model that captures general distribution of the next state. It models the next state distribution independent of the semantic information(which would be the current state). The likelihood ratio statistic is defined as

$$\text{LLR}(x) = \log \frac{p_\theta(s'|s)}{p_{\theta_0}(s')} = \log \frac{p_\theta(s'|s)_B p_\theta(s'|s)_S}{p_{\theta_0}(s')_B p_{\theta_0}(s')_S},$$

Assuming that both models capture the background information equally well, that is $p_\theta(s'|s)_B \approx p_{\theta_0}(s')_B$, and (ii) $p_\theta(s'|s)_S$ is more peaky than $p_{\theta_0}(s')_S$, as the former is trained on data containing semantic information, while the latter model θ_0 is trained using data with noise perturbations. Then, the likelihood ratio can be approximated as

$$\text{LLR}(x) \approx \log p_\theta(s'|s)_S - \log p_{\theta_0}(s')_S.$$

After taking the ratio, the likelihood for the background component ($(s'|s)_B$ and $(s')_B$) cancels out, and only the likelihood for the semantic component ($(s'|s)_S$ and $(s')_S$) remains. This produces a *background-contrastive* score that captures the significance of the semantics compared with the background model.

3.4 Positive Unlabeled Learning (PU Learning)

Both supervised reward learning and GAIL suffer from a shared misclassification of agent experience. In supervised reward learning, the reward model $r_\phi(s)$ is trained only on annotated states $\mathcal{D}_s = \{(s_i, r_i)\}_{i=1}^{N_s}$, leaving large regions of the state space uncovered. When the RL policy explores these regions, it exploits the reward model’s errors to achieve high pseudo-reward through unintended behaviors. This phenomenon is known as *reward delusion*. In GAIL, the discriminator D_θ is trained under a positive-negative (PN) objective that treats all agent rollouts as negative data. This is valid early in training when the agent behaves randomly, but as the policy improves and begins completing the task successfully, some rollouts are genuinely positive (successful). Forcing these into the negative class causes the discriminator to overfit to task-irrelevant features to maintain separation between expert and agent data. In both cases, the root cause is the same: agent experience is a *mixture* of successes and failures, not a clean negative set. Treating unlabeled mixtures as negative data is semantically wrong, and PU learning directly corrects this misclassification without requiring any additional labels or assumptions.

The key insight of PU learning is that the true negative risk $R_g^0(\mathcal{D}_n)$, which requires labeled negative data, can be decomposed entirely in terms of positive data $\mathcal{D}_p$ and unlabeled data $\mathcal{D}_u$ via the identity:

$$(1 - \eta) R_g^0(\mathcal{D}_n) = R_g^0(\mathcal{D}_u) - \eta R_g^0(\mathcal{D}_p),$$

where $\eta = P(Y = 1)$ is the positive class prior, and $R_g^y(\mathcal{D}) = \mathbb{E}_{\mathcal{D}}[\ell(g(x), y)]$ is the risk operator for class y . Substituting this into the standard PN risk $R_g^{pn} = \eta R_g^1(\mathcal{D}_p) + (1 - \eta) R_g^0(\mathcal{D}_n)$ yields the PU risk:

$$R_g^{pu}(\mathcal{D}_p, \mathcal{D}_u) = \eta R_g^1(\mathcal{D}_p) - \eta R_g^0(\mathcal{D}_p) + R_g^0(\mathcal{D}_u).$$

However, the empirical estimate $\hat{R}_g^{pu}$ can become negative when using complex function approximators such as deep networks, causing overfitting. Kiryo et al. (2017) resolve this by introducing a non-negative correction with slack $\beta \geq 0$:

$$\tilde{R}_g^{pu}(\mathcal{D}_p, \mathcal{D}_u) = \eta \hat{R}_g^1(\mathcal{D}_p) + \max\left(-\beta, \hat{R}_g^0(\mathcal{D}_u) - \eta \hat{R}_g^0(\mathcal{D}_p)\right),$$

which bounds the estimation error even for highly expressive models. In the PURL framework, expert demonstrations $\mathcal{D}_{\pi_e}$ (or annotated states $\mathcal{D}_s$) serve as the positive set, and the agent replay buffer $\mathcal{D}_{\pi}$ serves as the unlabeled set, directly plugging into this estimator to train a discriminator with the correct success-vs-failure semantics.

4 Methodology

4.1 Objective

We assume access to small quantities of expert *state-only* demonstration trajectories from a target agent,

$$\mathcal{E}_e = \{\tau_i\}_{i=1}^k, \quad \tau_i = \{s_0, s_1, \dots, s_N\},$$

along with a set of large quantities of task-agnostic *state-action* trajectories collected from imitator-like agents,

$$\mathcal{E}_n = \{T_j\}_{j=1}^l, \quad T_j = \{(s_0, a_0), (s_1, a_1), \dots, (s_N, a_N)\}.$$

Given these trajectories, the aim is to learn a policy that would enable the imitator agent to imitate/follow the trajectories from the expert dataset.

Access to a simulator and information regarding the expert trajectories is not available to us.

The primary objective can be written as minimizing the divergence between the policy state transition

distribution and the expert state transition distribution, while satisfying the constraints:

$$\min_{\pi} D_{\text{KL}}\left(\pi_E(a \mid s) \parallel \pi_{\theta}(a \mid s)\right)$$

where $d^{\pi}(s', s)$ is the state-transition distribution of the imitator policy, and $d^E(s', s)$ is the state-transition distribution of the expert. γ is the discounting factor, $T(s_t, a_t)$ is the transition distribution, $\pi(s_t)$ is the policy function. $D_{\text{KL}}(\cdot \parallel \cdot)$ is the KL divergence between two distributions.

4.2 Approach

1. **Sub-Trajectory State-Flow Identification.** Reinforcement Learning maximises the expected return from a given state. Behaviour Cloning primarily depends on state transitions to learn a policy. To extract *trajectory-level* information, we construct *state sequences* (sub-trajectories) from the dataset. We call these *state-flows* for use in subsequent steps.
2. **State importance estimation.** We do not directly have access to the state-transition distributions. We identify **overlap** between the *expert like state-flows* within the *task-agnostic unlabelled dataset (imitator dataset)*, and assign importance weights to these states-flows.

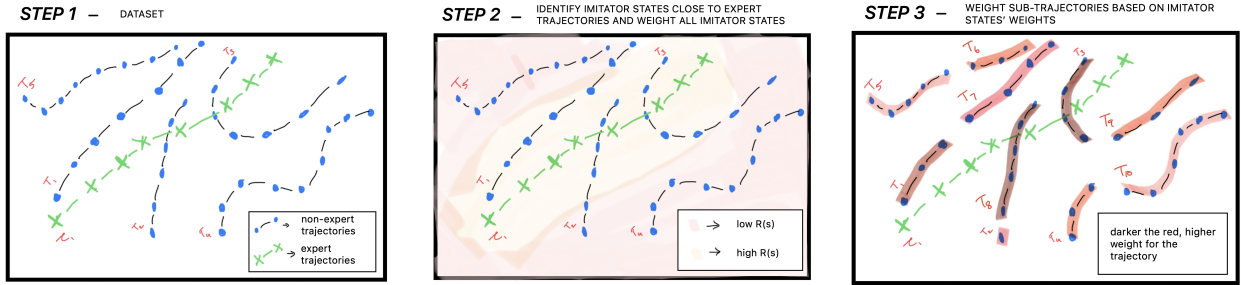


Figure 1: State Importance Estimation

3. **Weighted behaviour cloning.** Using the *imitator state-action* dataset and the *weights* learned for state-flows, we learn a weighted behaviour cloning policy Kim et al. (2022b), Yan et al. (2023a) optimizing the objective above.

4.2.1 Sub-Trajectory State-Flow Identification.

The overall motion of a trajectory can be represented as a sequence of states within a specified look-ahead horizon. We define a state-flow with the look-ahead distance be denoted by L , and the sequence length of the state-flow by N , where

$$N \leq L.$$

We sample N time indices within the interval $[t, t + L - 1]$, denoted as

$$\{j_1, j_2, \dots, j_N\}, \quad \text{where } j_1 < j_2 < \dots < j_N.$$

The corresponding state-flow is given by

$$(s_{j_1}, s_{j_2}, \dots, s_{j_N}).$$

Since we have much more information about a sequence of states than about a pair of states, this can be seen as an equivalent to capturing the direction of motion or a potential trajectory estimate. This allows discrimination of trajectories as a whole rather than just using independent transitions.

The idea was originally motivated as an alternative to conventional state-transition modeling in the presence of data sparsity. Given that many learning algorithms are capable of generalizing from as few as one to five trajectories, it becomes essential to maximize the informational content extracted from each observed trajectory.

Accordingly, this approach seeks to mitigate the effects of *data sparsity* by constructing multiple plausible state flows from a limited set of expert demonstrations, thereby enriching the effective training distribution without requiring additional data collection.

4.2.2 State Importance Estimation

Existing literature primarily relies on discriminating between the expert dataset and the non-expert dataset to estimate a reward score for each transition Kim et al. (2022b) Ma et al. (2022b) Yan et al. (2023a).

$$R(s, s') = \log \left(\frac{d^E(s, s')}{d^I(s, s')} \right)$$

However, there are two major issues with these methods:

- They use this method under the assumption that if $d_E(s) > 0$, then $d_{NE}(s) > 0$, which implies that states are present and overlap within these two datasets. However, this is a very strong assumption that must be relaxed. Kim et al. (2022b) Ma et al. (2022b)
- The discrimination often treats the problem as a positive-negative binary classification. However, there are sequences and transitions within the imitator dataset that are useful to identify. Yan et al. (2023a)

These two can lead to estimation errors Chen et al. (2020) seen in Fig.2

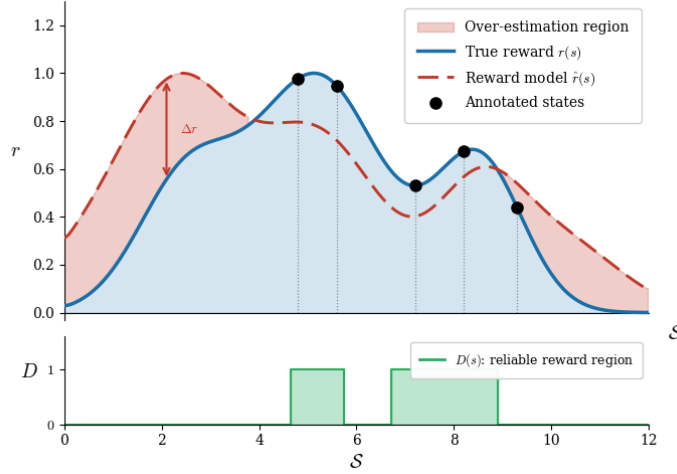


Figure 2: The difference in rewards between the true rewards and estimated rewards caused by binary discrimination between expert and non-expert datasets. Ideally, only states in green region can be given rewards with high confidence.

Since we do not have access to $d^E(s, s')$ and $d^I(s, s')$, we must estimate them.

Gaussian Mixture Models : We would ideally want to model a joint distribution model between the current state and next state. We can maximize the likelihood of fitting multiple Gaussian Models each weighted to different values to represent the data distribution.

However, this does not scale well to higher dimensions and the volume of data that we have. It would end up over generalizing the distribution that we require.

Instead, we model the state transition dynamics using a Gaussian Mixture Model (GMM), which captures the feasibility of transitioning to a next state given the current state.

$$\begin{aligned}
 R(s, s') &= \log \frac{d^E(s', s)}{d^N(s', s)} \\
 &\approx \log \frac{P^E(s' | s)}{P^N(s' | s)} \\
 &= \log P^E(s' | s) - \log P^N(s' | s)
 \end{aligned}$$

$$P(s' | s) = \sum_{k=1}^K w_{\theta}^k(s) \mathcal{N}(s' | \mu_{\theta}^k(s), \sigma_{\theta}^k(s)) = f(s)$$

- $w_{\theta}^k(s)$: The mixing coefficient of the k -th Gaussian component, which is dependent on the state s .
- $\mu_{\theta}^k(s)$: The mean of the k -th Gaussian component, dependent on the state s .
- $\sigma_{\theta}^k(s)$: The variance of the k -th Gaussian component, dependent on the state s .
- $\mathcal{N}(\cdot)$: The probability density function of the Normal (Gaussian) distribution.
- The mean, mixing coefficient, and variance are parameterized by θ .

We would want to maximize the likelihood of the above distribution given the data:

$$\min_{\theta} -\log(P_{\theta}(s'|s))$$

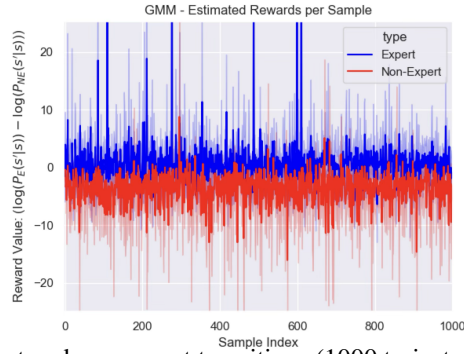


Figure 3: Rewards for expert and non-expert transitions (1000 trajectories each) for the hopper agent.

Instead of predicting the next state conditioned only on the current state, we reformulate the problem with state-flows. Specifically, given the first k states, we aim to predict the remaining $N - k$ states:

$$(s_{j_{k+1}}, s_{j_{k+2}}, \dots, s_{j_N}) = f(s_{j_1}, s_{j_2}, \dots, s_{j_k}).$$

The function f is parameterized by a Neural Network with a stochastic output head. The model predicts a conditional distribution over the future states represented by a Gaussian Mixture output head:

$$p_{\theta}(s_{j_{k+1}}, \dots, s_{j_N} \mid s_{j_1}, \dots, s_{j_k}).$$

The model is trained by minimizing the negative log-likelihood (NLL) of the target sequence given the input sequence:

$$\mathcal{L}_{\text{NLL}} = -\mathbb{E}_{D_{SF}} [\log p_{\theta}(s_{j_{k+1}}, \dots, s_{j_N} \mid s_{j_1}, \dots, s_{j_k})].$$

where D_{SF} is a state-flow dataset.

For representational purposes,

- the conditional is written in shorthand using $\bar{s}_1^k = (s_{j_1}, s_{j_2}, \dots, s_{j_k})$
- the predicted output states are written as $\bar{s}_{k+1}^N = (s_{j_{k+1}}, s_{j_{k+2}}, \dots, s_{j_N})$

Consider a sequence length of 3. For each sequence of length 3, consider a separation index $k = 2$. The standard model estimates the conditional probability:

$$g(s_2 \mid s_0, s_1).$$

In contrast, the model trained using partially masked inputs. For instance, the model estimates:

$$\bar{g}(s_2 \mid s_0), \quad \bar{g}(s_2 \mid s_1).$$

This masking strategy allows the generation of *multiple training datapoints* while *retaining the partial structure* of the trajectory, thereby preventing complete loss of contextual information during training. The estimated model is also more *generalised* due to fewer conditions. As the number of conditions decreases, the distribution approaches the joint distribution of a state-flow. In this way, state flows enable the modelling of complex distributions.

To estimate the final background log-likelihood, we aggregate predictions across different masking configurations. Let the likelihood estimates corresponding to k different masks be:

$$\{p_1, p_2, \dots, p_k\}.$$

Assumption. Each masking configuration is assumed to be equally likely during training.

Under this assumption, the effective likelihood is computed as the average:

$$\bar{p} = \frac{1}{k} \sum_{i=1}^k p_i.$$

The corresponding background log-likelihood can then be written as:

$$\log \bar{p} = \log \left(\frac{1}{k} \sum_{i=1}^k p_i \right).$$

Positive Unlabelled Learning A modified objective is created for the density estimator to improve the discrimination. This would particularly be useful in more *complex environments* and *sparse*

datasets.

We want to prioritize expert samples. Simultaneously, we want to penalize regions not in the expert dataset. In this way, we avoid assumptions about the non-expert dataset and score it specifically based on the expert dataset.

The objective for PU regularization is given by Xu and Denil (2019). Let $(x_{i1}^k, x_{i(k+1)}^N)$ and $(x_{j1}^k, x_{j(k+1)}^N)$ be samples from the dataset. Define a convex combination:

$$\bar{x}_1^k = \lambda x_{i1}^k + (1 - \lambda)x_{j1}^k, \quad \bar{x}_{k+1}^N = \lambda x_{i(k+1)}^N + (1 - \lambda)x_{j(k+1)}^N, \quad \lambda \sim \text{Uniform}(0, 1).$$

The corresponding PU loss, modeling the conditional density of the second block given the first, is:

$$\mathcal{L}_{\text{PU}}(\bar{x}_1^k, \bar{x}_{k+1}^N) = -\log \left(\sum_{m=1}^K \pi_m(\bar{x}_1^k) \mathcal{N}(\bar{x}_{k+1}^N \mid \mu_m(\bar{x}_1^k), \sigma_m^2(\bar{x}_1^k)) \right).$$

Thresholded Penalty for Non-Expert Sequences For non-expert samples (x_1^k, x_{k+1}^N) , we define a thresholded loss:

$$\mathcal{L}_{\text{thr}}(x_1^k, x_{k+1}^N) = \max \left(-\log \left(\sum_{m=1}^K \pi_m(\bar{x}_1^k) \mathcal{N}(\bar{x}_{k+1}^N \mid \mu_m(\bar{x}_1^k), \sigma_m^2(\bar{x}_1^k)) \right) - \tau, 0 \right)$$

where $\tau > 0$ is a threshold hyperparameter.

So the overall objective for the density model is defined by three primary terms.

Overall Objective. The total loss for a single training sample is:

$$\mathcal{L} = \mathcal{L}_{\text{NLL}}(\bar{x}_1^k, \bar{x}_{k+1}^N) - \eta \mathcal{L}_{\text{PU}}(\bar{x}_1^k, \bar{x}_{k+1}^N) - \zeta \mathcal{L}_{\text{thr}}(\bar{x}_1^k, \bar{x}_{k+1}^N)$$

The final score assigned to each transition in a state-flow depends on both the length of the state-flow and the log-likelihood of the state-flow under the learned density estimate.

First, a normalized score is computed for a given state-flow $\bar{s}$ as:

$$\bar{w}(\bar{s}) = \frac{1}{\tau} \cdot \frac{\widehat{\log p}(s_{k+1:N} \mid s_{1:k}) - B(\hat{p}; D_E, D_{NE})}{\sigma(\hat{p}; D_E, D_{NE})}, \quad (1)$$

where $B(\cdot)$ and $\sigma(\cdot)$ denote the baseline and standard deviation computed over the datasets D_E and D_{NE} , respectively.

The resulting scalar is exponentiated to obtain a positive weight:

$$w(\bar{s}) = \exp(\bar{w}(\bar{s})). \quad (2)$$

Since the state-flow $\bar{s}$ corresponds to a sub-trajectory, we interpret $w(\bar{s})$ as an estimate of the (unnormalized) return associated with its initial state. Under a uniform per-step reward assumption—analogueous to distributing Monte Carlo returns evenly across a trajectory—we propagate this value to individual transitions.

For a state-flow of length L , each transition at index $i \in \{1, \dots, L\}$ is assigned:

$$w_i = \frac{L - i}{L} \cdot w(\bar{s}). \quad (3)$$

4.2.3 Weighted Imitation

The primary objective is to match the learned policy π_θ to the expert policy π_E . This can be expressed as minimizing the Kullback–Leibler divergence:

$$\min_{\theta} \text{KL}(\pi_E \parallel \pi_\theta)$$

$$\begin{aligned} \text{KL}(\pi_E \parallel \pi_\theta) &= \mathbb{E}_{\tau \sim \pi_E} \left[\log \frac{\pi_E(\tau)}{\pi_\theta(\tau)} \right] \\ &= \mathbb{E}_{\tau \sim \pi_E} \left[\log \frac{p(s_0) \prod_{t=0}^T \pi_E(a_t | s_t) T(s_{t+1} | s_t, a_t)}{p(s_0) \prod_{t=0}^T \pi_\theta(a_t | s_t) T(s_{t+1} | s_t, a_t)} \right] \\ &= \mathbb{E}_{\tau \sim \pi_E} \left[\sum_{t=0}^T \log \frac{\pi_E(a_t | s_t)}{\pi_\theta(a_t | s_t)} \right] \\ &= \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \pi_E} \left[\log \frac{\pi_E(a_t | s_t)}{\pi_\theta(a_t | s_t)} \right] \\ &= (1 - \gamma) \sum_{t=0}^T \mathbb{E}_{(s, a) \sim \rho_E^{(t)}} \left[\log \frac{\pi_E(a | s)}{\pi_\theta(a | s)} \right] \end{aligned}$$

$$\begin{aligned}
&= \mathbb{E}_{(s,a) \sim \rho_E} \left[\log \frac{\pi_E(a|s)}{\pi_\theta(a|s)} \right] \\
&\propto \mathbb{E}_{(s,a) \sim \rho_E} \left[\log \frac{\rho_E(a|s)}{\pi_\theta(a|s)} \right]
\end{aligned}$$

where the discounted occupancy measure is defined as:

$$\rho_E(s, a) = (1 - \gamma) \sum_{t=0}^T \gamma^t \Pr(s_t = s, a_t = a \mid \pi_E)$$

and satisfies:

$$\rho_E(a|s) = \pi_E(a|s)$$

$$\boxed{\text{KL}(\pi_E \parallel \pi_\theta) \propto \text{KL}(\rho_E \parallel \pi_\theta) = \mathbb{E}_{(s,a) \sim \rho_E} \left[\log \frac{\rho_E(a|s)}{\pi_\theta(a|s)} \right]}$$

Now, the modified objective is:

$$\min_{\theta} \text{KL}(\rho_E \parallel \pi_\theta) = \min_{\theta} \mathbb{E}_{(s,a) \sim \rho_E} \left[\log \frac{\rho_E(a|s)}{\pi_\theta(a|s)} \right]$$

Rewriting using a non-expert distribution. Direct sampling from ρ_E may not always be feasible. Instead, we rewrite the expectation using a uniform non-expert distribution ρ_{NE} :

$$= \min_{\theta} \mathbb{E}_{(s,a) \sim \rho_{NE}} \left[\frac{\rho_E(s, a)}{\rho_{NE}(s, a)} \log \frac{\rho_E(a \mid s)}{\pi_\theta(a \mid s)} \right].$$

Expert density approximation via state flows. We approximate the expert state-action distribution $\rho_E(s, a)$ using a log-likelihood ratio (LLR) score derived from state flow models. This score reflects how likely a trajectory segment is under the expert distribution.

After normalization, this yields:

$$\rho_E(s, a) \approx \frac{\exp(\bar{\omega}(s, a))}{Z},$$

where

$$\bar{\omega}(s, a) = f(\bar{s}),$$

and Z is the partition function and $\bar{s}$ is a state flow, and $f(\cdot)$ is a density function of the state-flow.

Substituting this into the objective:

$$= \min_{\theta} \mathbb{E}_{(s,a) \sim \rho_{NE}} \left[\frac{\exp(\bar{\omega})}{Z \rho_{NE}(s, a)} \log \left(\frac{\frac{\exp(\bar{\omega})}{Z}}{\pi_{\theta}(a | s)} \right) \right].$$

Expanding the logarithm:

$$= \min_{\theta} \mathbb{E}_{(s,a) \sim \rho_{NE}} \left[\frac{\exp(\bar{\omega})}{Z \rho_{NE}(s, a)} (\bar{\omega} - \log Z - \log \pi_{\theta}(a | s)) \right].$$

Simplification. Grouping terms and isolating those dependent on θ , we obtain:

$$= \min_{\theta} k_1 \mathbb{E}_{(s,a) \sim \rho_{NE}} [-\exp(\bar{\omega}) \log \pi_{\theta}(a | s)] - k_2,$$

where k_1 and k_2 are constants independent of θ .

Dropping constants, the objective simplifies to:

$$\min_{\theta} -\mathbb{E}_{(s,a) \sim \rho_{NE}} [\exp(\bar{\omega}) \log \pi_{\theta}(a | s)].$$

Interpretation. This corresponds to a weighted behavioral cloning objective, where each sample is weighted by $\exp(\bar{\omega})$ which is a function of the state-flow, i.e., an exponentiated, normalized LLR score indicating how likely the trajectory segment is under the expert distribution.

Assumptions.

1. **Uniform non-expert distribution:** The non-expert dataset $\rho_{NE}(s, a)$ is assumed to be approximately uniform, allowing it to act as a proposal distribution for importance weighting.

5 Experiments and Observations

Building upon the theoretical framework, we conducted systematic experiments to calibrate the density estimation component and to derive an appropriate weighting formulation for behavioral cloning. Empirical observations from these studies guided an iterative refinement process; however, the resulting approach has not yet achieved consistent or reliable performance.

5.1 Normalization in Density Estimation

In an ideal setting, the learned reward distribution should reflect the structure of the underlying data: expert trajectories should receive the highest rewards, followed by medium-quality agents, and finally random or suboptimal behaviour.

However, a key limitation is observed in Fig. 4. While the density estimator is able to distinguish between these categories, the resulting scores in logarithmic space are extremely poorly scaled. In practice, the values span a wide range. Such a scale is problematic when used in exponentiation-based weighting schemes.

Direct exponentiation of these values leads to severe numerical instability. Large positive values dominate, while large negative values collapse towards zero, resulting in exploding or vanishing gradients. This makes the optimization highly unstable and ineffective.

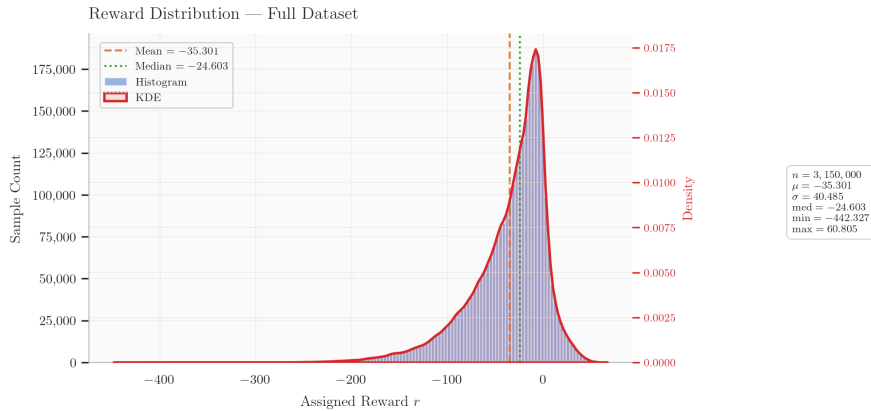


Figure 4: Logarithmic score distribution of generated samples in the Hopper dataset. The values span a wide range (approximately -2200 to 65) and are concentrated in the negative region, which makes direct exponentiation numerically unstable.

While the issue of scaling remains significant, Fig. 5a shows a clear separation of the data into three distinct regions. In particular, samples from the expert dataset consistently receive higher scores than those from the medium and random datasets. This indicates that the density estimator is able to effectively distinguish between different data distributions.

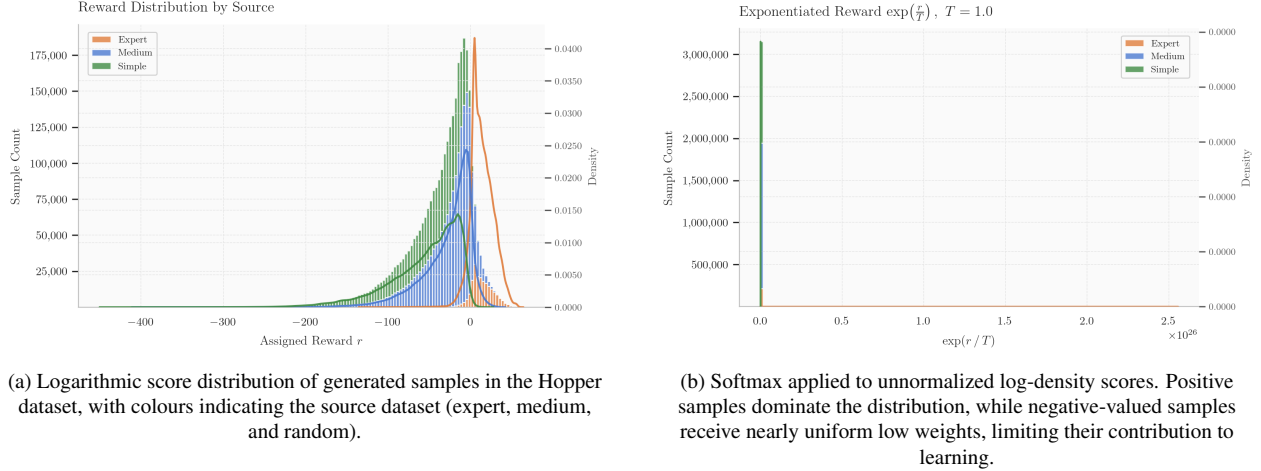


Figure 5: Effect of log-density scaling on sample weighting.

When operating on batches, the exponential normalization results in the weight being a softmax over the assigned scores. As seen in Fig.5b, without normalisation, most of the scores are pushed to near-zero values from expert, medium and random agents. Behaviour Cloning with these weights results in no learning because the most useful transitions are assigned a score of 0.

The solution for this would be to apply a baseline subtraction, and scaling it to a range of variance 1.

$$\bar{w}(\bar{s}) = \frac{1}{\tau} \frac{\log p(s_{k+1:N} | s_{1:k}) - B(\hat{p}; D_E, D_{NE})}{\sigma(\hat{p}; D_E, D_{NE})}$$

where, $B(\hat{p}; D_E, D_{NE})$, $\sigma(\hat{p}; D_E, D_{NE})$ are functions of the density estimate applied on the dataset available to us and τ is the temperature parameter.

For the $\sigma(\cdot)$ function, a standard deviation-based scaling is sufficient to bring the values within a suitable range. In contrast, the $B(\cdot)$ function plays a more critical role, as it serves as a baseline that determines the relative importance of trajectories during learning. Specifically, it controls how negative values are suppressed and how positive values are emphasized.

The choice of $B(\cdot)$ depends on the composition of the dataset. When expert data is limited compared to medium and random data, the baseline should be set above the mean by 1 or 2 standard deviations to ensure that higher-quality trajectories receive greater emphasis. Alternatively, $B(\cdot)$ can be estimated directly from the expert dataset by selecting a lower bound of the expert score distribution as the baseline.

These observations highlight the need for proper normalization prior to exponentiation. When the mean of the non-expert dataset is used as $B(\cdot)$ and its standard deviation is used as $\sigma(\cdot)$, the resulting normalized scores exhibit a controlled range.

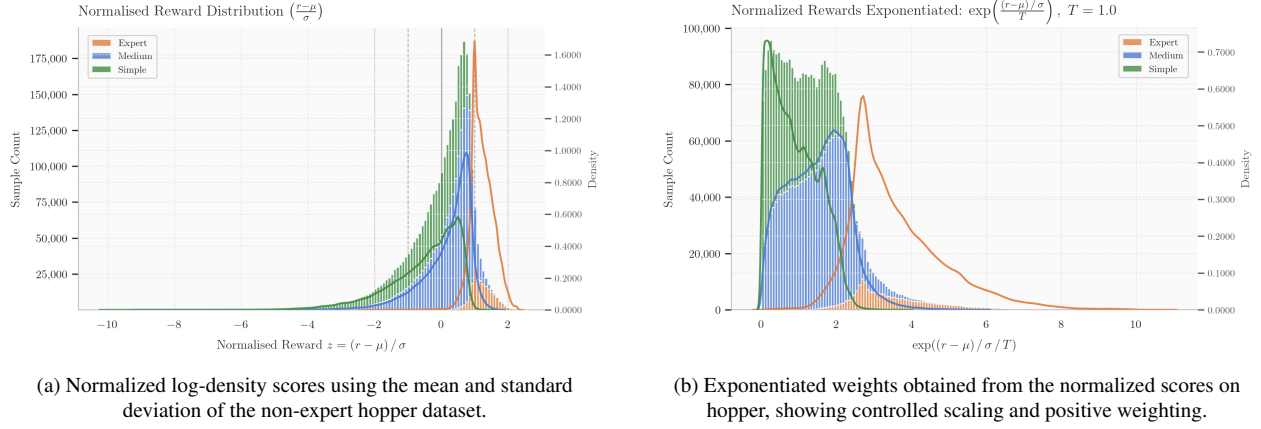


Figure 6: Effect of normalization and exponentiation on log-density scores.

As shown in Fig. 6, this normalization leads to well-scaled values before exponentiation and produces stable, positive weights after exponentiation, which are suitable for use in mean squared error (MSE) based Behaviour Cloning. Since the transformation in $\bar{w}(\bar{s})$ consists only of shifting and scaling operations, the overall structure of the distribution is preserved. Fig. 6a also shows us that while it is able to distinguish between the data, it is mostly concentrated around 1. This would be something that will be influenced by the choice of $B(\cdot)$ and the temperature parameters. However, the exponentiated normalised score in Fig. 6b ranges from 0 to 10, providing a strong distinction between the three regions.

When the original rewards in the dataset are examined, the resulting distribution aligns with the expected pattern. High-quality trajectories, primarily from the expert dataset, are concentrated toward the higher reward region, while medium and random trajectories occupy the lower end.

For state-flow sequences of length 7, the reward distribution follows this expected trend, as shown in Fig. 7. However, for longer sequences (e.g., length 25), the distribution exhibits greater separation, allowing clearer distinction between expert and non-expert trajectories. Especially the pattern seen in Fig. 7b is the kind of separation we desire. The temperature and baseline terms are parameters used to control the normalisation.

The distributions presented above correspond to a single environment. To assess consistency across different settings, the exponentiated score distributions for the remaining environments—Ant, HalfCheetah, and Walker2d—are shown in Fig. 8.

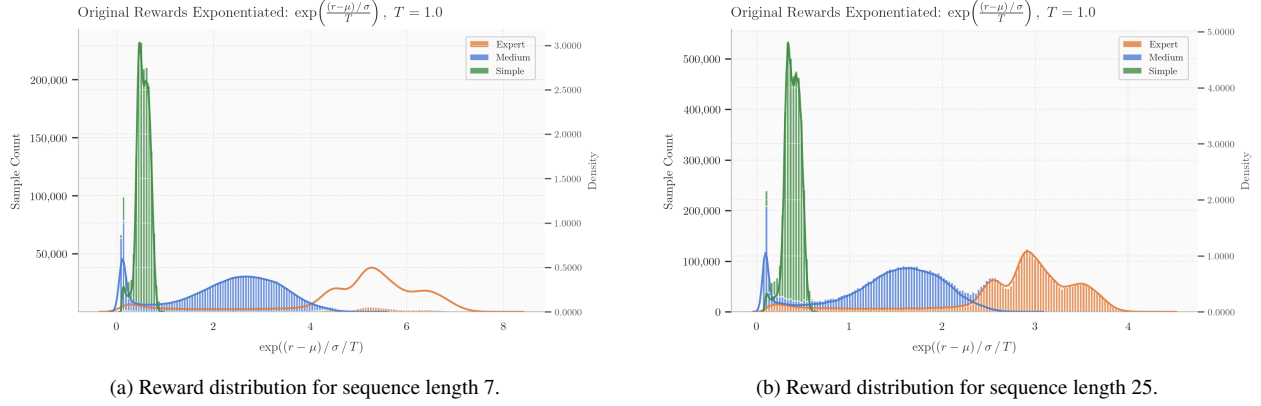


Figure 7: Comparison of original reward distributions for halfcheetah for different state-flow sequence lengths. Longer sequences result in improved separation between expert and non-expert trajectories, particularly for the expert trajectories.

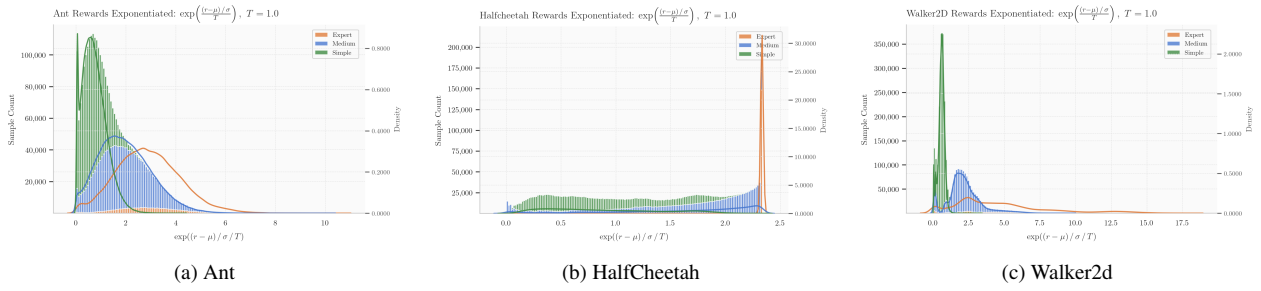


Figure 8: Exponentiated score distributions across different environments. The trends remain consistent, showing controlled scaling and clear separation between expert, medium, and simple trajectory qualities.

5.2 Behaviour Cloning works only on Hopper

5.2.1 Performance on Hopper

All density estimates exhibit a clear separation between expert and non-expert trajectories, indicating that the learned distributions effectively capture the underlying structure of the data. Building on this, Behaviour Cloning (BC) was implemented by optimizing the objective defined in Section 4.2.3.

The results in the Hopper environment are encouraging: the learned policy is able to capture long-horizon expert behaviour and achieves performance comparable to established baselines such as SMODICE, TAILO, and LobsDICE.

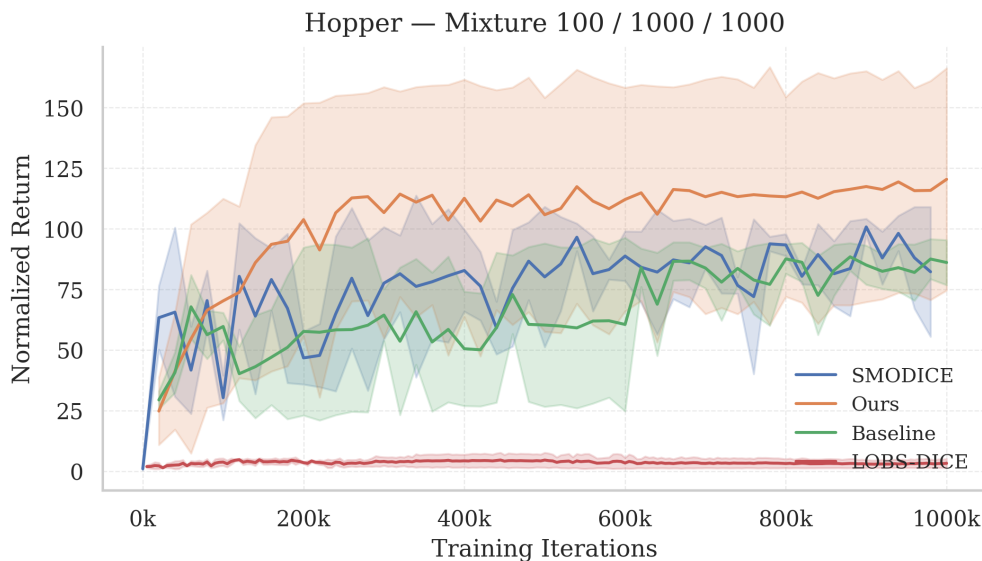


Figure 9: Performance comparison of Behaviour Cloning with baseline methods in the Hopper environment.

However, Figure 9 also highlights an important observation: standard Behaviour Cloning, even without weighting, performs competitively—and in some cases surpasses methods such as SMODICE in this relatively simple environment. This suggests that Hopper may not be sufficiently challenging to fully evaluate the advantages of the proposed approach. [Note: In LobsDICE all runs lead to subpar performance on all the Gymnasium environments which were v4 and v5 unlike the original paper, and I was not able to recreate results from the original paper. This could be rising from bad hyperparameter tuning]

Consequently, it becomes necessary to assess performance in more complex and diverse environments, beginning with HalfCheetah and extending to Walker2d and other higher-dimensional control tasks, where the limitations of naive Behaviour Cloning are more likely to manifest.

5.3 Performance on other environments

However, as illustrated in Figure 10, the policy in the HalfCheetah environment consistently learns from suboptimal behaviour and fails to surpass a normalized reward of 0, which corresponds to the performance of basic (simple) trajectories. Despite continued training, the agent does not exhibit meaningful improvement beyond this baseline. Notably, this trend is not isolated to HalfCheetah but is consistently observed across multiple environments, indicating a systematic limitation of the current approach rather than an environment-specific issue.

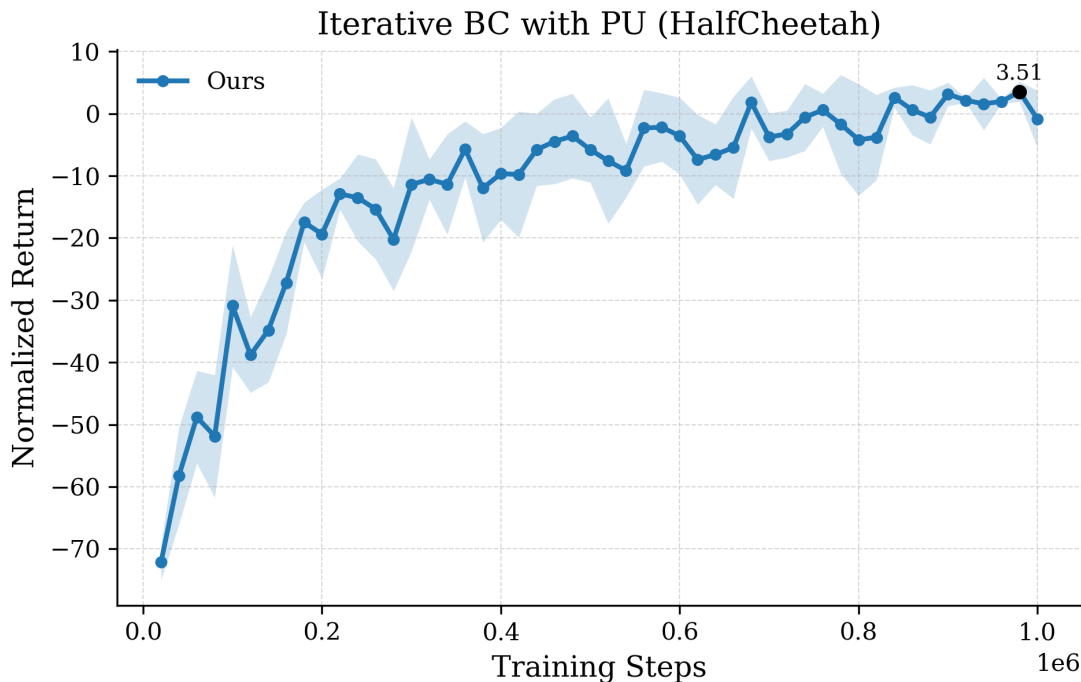


Figure 10: Training performance in the HalfCheetah environment, showing that the learned policy fails to exceed the baseline normalized reward of 0, corresponding to simple trajectories.

This behaviour can plausibly be attributed to several underlying factors:

1. **Decoupled training of density estimation and policy learning:** The independent training of the density model and the Behaviour Cloning policy may introduce unfavorable initial conditions. Since the policy relies on fixed weights derived from the density estimates, any inaccuracies early on can propagate through training. An iterative or joint optimization procedure could allow the weighting mechanism to adapt alongside policy improvement, potentially mitigating this issue.
2. **Poor separation in exponentiated score distributions:** As observed in the density plots, the exponentiated scores do not exhibit sufficient spread. This results in weak discrimination between expert and non-expert trajectories during training, particularly in datasets dominated

by simple and medium-quality trajectories. Consequently, the weighting mechanism fails to provide a strong learning signal. This can be a problem attributed to the density estimates.

3. **Suboptimal baseline selection:** The choice of baseline in the weighting scheme may be misaligned, leading to relatively high weights being assigned to poor-quality trajectories. This diminishes the relative importance of expert demonstrations and can bias the policy toward suboptimal actions.
4. **Distributional shift in Behaviour Cloning:** Owing to its supervised learning formulation, Behaviour Cloning is inherently susceptible to covariate shift. Errors made by the policy compound over time as it encounters states that deviate from the training distribution, limiting its ability to recover and improve beyond suboptimal behaviour.

5.3.1 Iterative Training

This aims to address issue 1 in 4. Currently, Behaviour Cloning (BC) training is performed only after the density model has fully converged. While these two components are mathematically independent, this sequential training procedure can lead to poor initialization of the policy. In particular, the policy transitions abruptly from a random initialization to one trained on a fully reweighted dataset, which can negatively impact optimization.

To mitigate this, we propose an *iterative training scheme* where density estimation and policy learning are interleaved. This allows both models to co-evolve: the density estimator gradually refines the reward signal, while the policy adapts incrementally, leading to improved gradient guidance and more stable learning.

The full procedure is described in Algorithm 1. However, the algorithm has little to no impact in learning. All the results following have this algorithm in place.

5.3.2 Positive Unlabelled Learning

Addressing issue 2 in 4. As seen in Fig. 11, this formulation based on a just the expert data and a single model suffices, enabling a unified density-based discrimination framework.

As seen in Fig.12, we notice that as the number of data samples increase, the models are still able to distinguish between the expert and non-expert. However, the masked model is able to distinguish with access to lesser information, leading to a more general model.

The results have indicated that density estimation is working and is able to distinguish data points. However, the Behaviour Cloning results still show little to no improvements.

This indicates potential failure in the Behaviour Cloning rather than in density estimation.

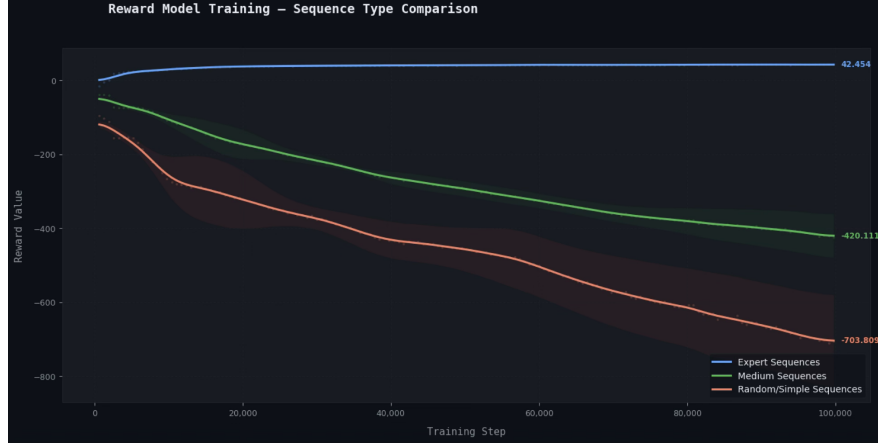
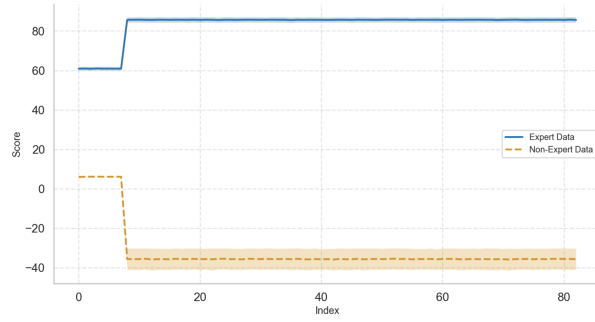
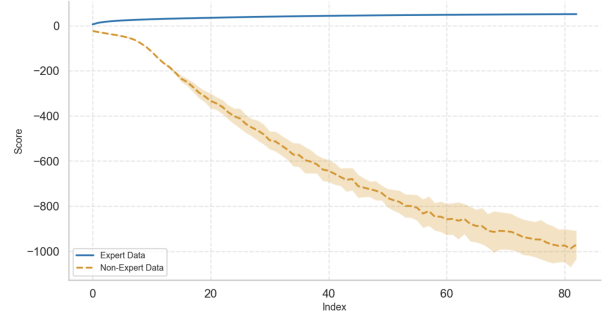


Figure 11: $\bar{w} = \log P(\bar{s}' | \bar{s})$, where $\bar{s}'$ denotes the predicted sequence and $\bar{s}$ the conditioned sequence of states. With PU learning, the density model effectively distinguishes between expert, medium, and simple data after constructing state flows.



(a) Samples from the hopper dataset scored using the **masked model**. Expert and non-expert transitions are clearly separated.



(b) Samples scored using the **unmasked model**. A similar separation between expert and non-expert transitions is observed.

Figure 12: Comparison of reward scoring on hopper dataset samples. X-axis represents the number of samples used to train the density model with PU learning. The X-axis shows the increasing number of training steps with size 1000 steps per index.

5.3.3 Behaviour Cloning with Original Data

The dataset obtained from Minari provides access to the original rewards from the environment. We treat these rewards as state-associated signals and use them to construct trajectory-level weights. Since our objective is to approximate the likelihood and long-term return of trajectories, we consider state-flows of fixed length $T = 25$ and define weights using the cumulative reward (i.e., value function) from each timestep onward.

Formally, for a flow of length T , the weight assigned to timestep i is:

$$w_i = \sum_{k=i}^T r_k, \quad \forall i \in \{1, 2, \dots, T\}. \quad (4)$$

For a batch of B trajectories (or flows), each of length T , we obtain a total of $N = B \times T$ state-action pairs. Using these weights, we train a behaviour cloning policy by minimizing a weighted negative log-likelihood objective.

Per-timestep weighted objective. We assign weights to every state-action pair in the trajectory:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{j=1}^B \sum_{i=1}^T w_i^{(j)} \cdot \mathcal{L}_{\text{BC}}\left(\pi_{\theta}(a_i^{(j)} \mid s_i^{(j)})\right). \quad (5)$$

Initial-state weighted objective. Alternatively, if the weight w_i accurately captures long-term performance, it may suffice to apply supervision only at the initial state of each trajectory:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{j=1}^B w_1^{(j)} \cdot \mathcal{L}_{\text{BC}}(\pi_{\theta}(a_1^{(j)} \mid s_1^{(j)})). \quad (6)$$

In principle, this formulation is consistent with prior approaches such as SMODICE and LobsDICE, where occupancy measures are reweighted using return-based estimates. However, in practice, both variants exhibit limited performance gains. Even with trajectory lengths of 25 and return-based weighting, the policy struggles to learn effectively.

This suggests that the primary limitation lies in the Behaviour Cloning objective rather than the density estimation. In particular, the density model appears sufficiently expressive to produce meaningful and well-separated distributions, while the policy fails to effectively leverage these signals during training.

The key factors contributing to this limitation include: (i) the choice of baseline used for normalization,

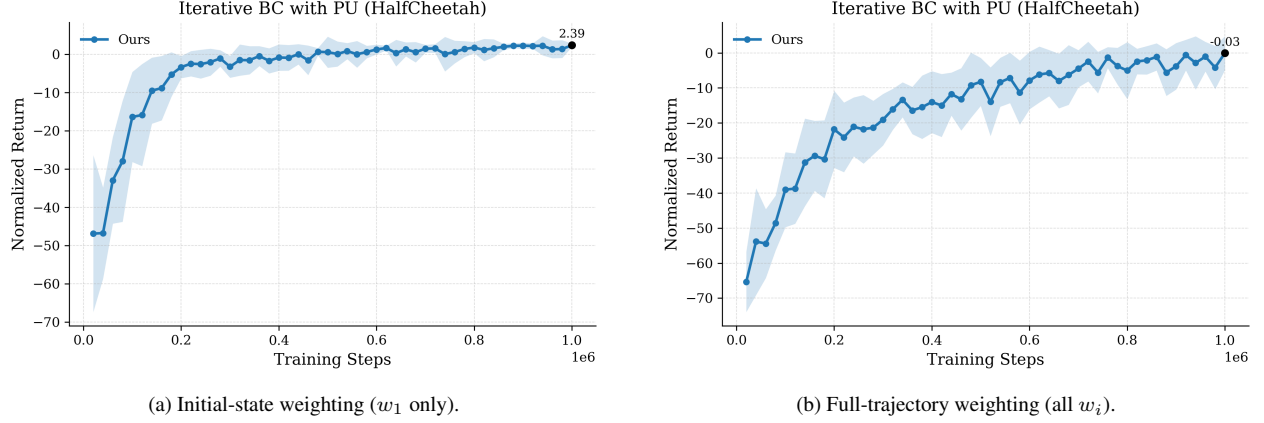


Figure 13: Behaviour Cloning performance under different weighting schemes. Both variants struggle to learn effectively despite return-based weighting.

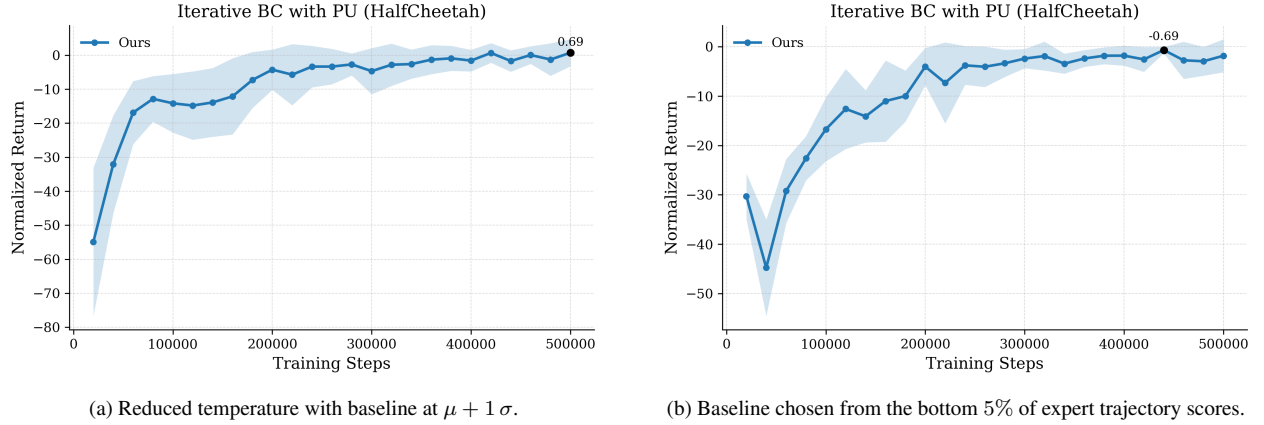


Figure 14: Effect of different baseline selection strategies on behaviour cloning performance. In Fig. 14a, lowering the temperature and using a baseline of $\mu + 1\sigma$ leads to occasional high-performing trajectories, with some rollouts achieving normalized scores above 50. However, this improvement is inconsistent across evaluations. In Fig. 14b, selecting a conservative baseline based on the worst-case expert trajectories produces a similar pattern: sporadic high returns without stable performance. These results suggest that while the policy is capable of discovering high-reward behaviours, it fails to reproduce them reliably. This inconsistency is likely due to distributional shift, where the learned policy deviates from the training distribution and cannot consistently recover favourable trajectories.

which can bias the relative weighting of trajectories; (ii) the temperature parameter controlling the spread of the weights, which directly affects gradient scaling and sensitivity to high-return samples; and (iii) the underlying distribution of expert, medium, and simple trajectories in the mixed dataset, which can lead to imbalance and insufficient emphasis on high-quality behaviour.

The qualitative results are shown in Figures 13a and 13b, corresponding to initial-state weighting and full-trajectory weighting respectively.

Across Fig. 14 both configurations, the presence of high-return trajectories indicates that the model has partially captured useful structure in the data. However, the lack of consistency highlights a fundamental challenge: performance is highly sensitive to the choice of baseline and temperature.

These hyperparameters are difficult to tune, as different settings trade off between stability and exploration of high-reward behaviours. Moreover, the observed variance suggests that even when good trajectories are present in the dataset, the learned policy struggles to generalize them reliably, reinforcing the impact of distributional shift in offline learning.

6 Points of Failure - Baseline, Distributional Shift in Behaviour Cloning

The preceding experiments —especially those conducted on the original rewards dataset — suggest that Behaviour Cloning faces significant challenges in learning an effective policy. In its current formulation, the method assigns weights to individual state transitions based on the exponentiated value function evaluated at the corresponding states. These weights are then used to guide the policy learning process.

As demonstrated in the previous experiments, the learning is potentially struggling because of

- **Temperature and Baseline Selection:** The weighting scheme is highly sensitive to these hyperparameters, which control gradient scaling and emphasis on high-return samples. Poor choices either over-amplify noise or underweight useful trajectories, making learning unstable and difficult to tune. This can potentially be addressed through Bayesian Parameter Tuning 7.2
- **Distributional Shift:** Although high-return trajectories exist and are occasionally reproduced, the policy fails to consistently recover them. Small deviations from the training distribution lead to compounding errors, preventing stable execution of high-quality behaviour.

Density estimation appears to function as intended, producing well-separated distributions between expert and imitation data. However, the learned policy is unable to effectively leverage these signals during training, resulting in suboptimal performance.

We outline several hypotheses to explain this limitation:

- **Multi-modality in expert trajectories.** When expert demonstrations are highly diverse, the underlying data distribution becomes multi-modal. Standard Behaviour Cloning implicitly assumes a unimodal mapping from states to actions, which can lead to averaging effects across modes.

Such averaging can produce trajectories that pass through high-cost or low-density regions that are not represented in the expert data. This hypothesis can be tested by constructing

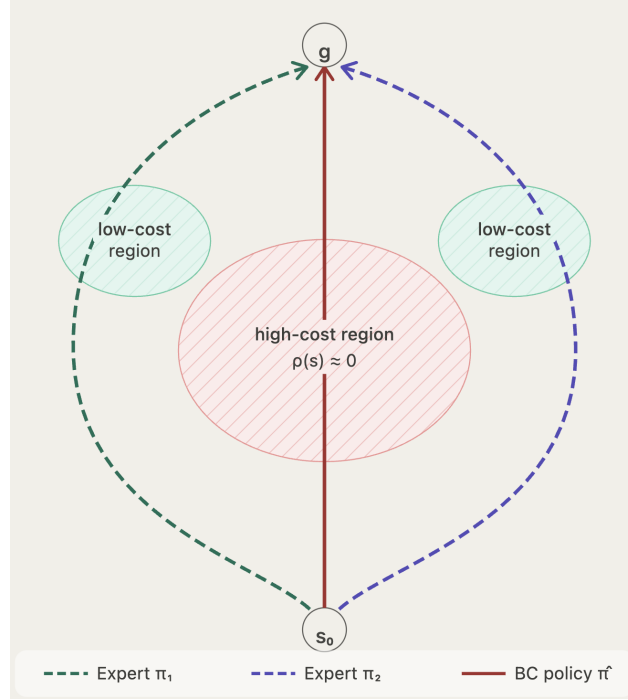


Figure 15: Illustration of multi-modal expert trajectories. Distinct modes in the expert data can lead Behaviour Cloning to interpolate between trajectories, potentially resulting in unrealistic or low-density paths.

controlled settings with clearly distinct expert trajectories and analyzing whether the learned policy interpolates between them.

- **Effectiveness of weighting relative to Behaviour Cloning.** It is important to evaluate how much improvement, if any, is obtained over a standard Behaviour Cloning baseline. In particular, we seek to determine whether the weighting mechanism meaningfully influences the learned policy, or whether it behaves similarly to unweighted supervised learning:

$$\mathcal{L}_{\text{BC}} = \mathbb{E}_{(s,a) \sim \mathcal{D}} [-\log \pi_{\theta}(a \mid s)], \quad (7)$$

compared to the weighted objective:

$$\mathcal{L}_{\text{weighted}} = \mathbb{E}_{(s,a) \sim \mathcal{D}} [w(s) (-\log \pi_{\theta}(a \mid s))]. \quad (8)$$

Empirical comparison is necessary to assess whether $w(s)$ provides a meaningful learning signal.

- **Trajectory selection via value-based weighting.** For a given state-action pair (s, a) , the action-value function $Q(s, a)$ represents the expected return over all possible future trajectories. Ideally, we are interested in identifying the trajectory that yields the highest return among these possible continuations. However, in continuous state-action spaces, explicitly enumerating

or recovering such trajectories is intractable. Moreover, directly learning a sharply peaked value function that isolates the optimal trajectory is challenging.

A potential approach is to approximate the optimal action-value function $Q^*(s, a)$ for each state-action pair and use it to score trajectories, rather than relying on empirical returns from individual trajectories. This could provide a more stable and informative weighting mechanism, potentially leading to improved policy learning.

References

- Abbeel, P. and A. Y. Ng (2004). Apprenticeship learning via inverse reinforcement learning. In *Proceedings of the twenty-first international conference on Machine learning*, pp. 1.
- Cao, Z., Y. Hao, M. Li, and D. Sadigh (2021). Learning feasibility to imitate demonstrators with different dynamics.
- Cao, Z. and D. Sadigh (2021). Learning from imperfect demonstrations from agents with varying dynamics.
- Chen, X., Z. Zhou, Z. Wang, C. Wang, Y. Wu, and K. Ross (2020). Bail: Best-action imitation learning for batch deep reinforcement learning.
- Dulac-Arnold, G., D. Mankowitz, and T. Hester (2019). Challenges of real-world reinforcement learning. *arXiv preprint arXiv:1904.12901*.
- Ho, J. and S. Ermon (2016). Generative adversarial imitation learning.
- Kim, G.-H., J. Lee, Y. Jang, H. Yang, and K.-E. Kim (2022a). Lobsdice: Offline learning from observation via stationary distribution correction estimation. In *Advances in Neural Information Processing Systems (NeurIPS) 35*.
- Kim, G.-H., J. Lee, Y. Jang, H. Yang, and K.-E. Kim (2022b). Lobsdice: Offline learning from observation via stationary distribution correction estimation.
- Konda, V. and J. Tsitsiklis (1999). Actor-critic algorithms. In S. Solla, T. Leen, and K. Müller (Eds.), *Advances in Neural Information Processing Systems*, Volume 12. MIT Press.
- Levine, S., Z. Popovic, and V. Koltun (2011). Nonlinear inverse reinforcement learning with gaussian processes. *Advances in neural information processing systems 24*.
- Ma, Y. J., A. Shen, D. Jayaraman, and O. Bastani (2022a). SmoDice: Versatile offline imitation learning via state occupancy matching. *International Conference on Machine Learning (ICML)*.

- Ma, Y. J., A. Shen, D. Jayaraman, and O. Bastani (2022b). SmoDice: Versatile offline imitation learning via state occupancy matching. *arXiv preprint arXiv:2202.02433* 1(2), 3.
- Mandyam, A., D. Li, A. Jones, D. Cai, and B. E. Engelhardt. Kd-birl: Kernel density bayesian inverse reinforcement learning. *Transactions on Machine Learning Research*.
- Ren, J., P. J. Liu, E. Fertig, J. Snoek, R. Poplin, M. A. DePristo, J. V. Dillon, and B. Lakshminarayanan (2019). Likelihood ratios for out-of-distribution detection.
- Sikchi, H., C. Chuck, A. Zhang, and S. Niekum (2024). A dual approach to imitation learning from observations with offline datasets.
- Sutton, R. S. and A. G. Barto (1998). *Reinforcement Learning: An Introduction* (1st ed.). Cambridge, MA: MIT Press. The second edition was published in 2018.
- Xu, D. and M. Denil (2019). Positive-unlabeled reward learning. *CoRR abs/1911.00459*.
- Yan, K., A. Schwing, and Y.-X. Wang (2023a). A simple solution for offline imitation from observations and examples with possibly incomplete trajectories. *Advances in Neural Information Processing Systems* 36, 589–623.
- Yan, K., A. G. Schwing, and Y.-X. Wang (2023b). Offline imitation from observation via primal wasserstein state occupancy matching. *arXiv preprint arXiv:2311.01331*.
- Yong, J. and X. Y. Zhou (1999). *Stochastic controls: Hamiltonian systems and HJB equations*, Volume 43. Springer Science & Business Media.

7 Appendix

7.1 Algorithm - Iterative Training

Algorithm 1 Density-Guided Policy Learning

- 1: **Input:** Expert dataset D_E , non-expert dataset D_{NE}
- 2: Construct state-flows F_E (with skips) from D_E
- 3: Construct state-flows F_{NE} (without skips) from D_{NE}
- 4: **function** TRAINDENSITY(F_E, F_θ)
- 5: Sample batch $X \sim F_E$
- 6: Update F_θ by maximizing:

$$\log F_\theta(x) - \eta \log F_\theta(\text{convex}(x_i, x_j)), \quad x, x_i, x_j \sim X$$

- 7: **return** F_θ
- 8: **end function**
- 9: **function** TRAINBC(F_{NE}, Π_Φ)
- 10: Sample batch $Y \sim F_{NE}$
- 11: Update Π_Φ by minimizing:

$$\sum_Y \frac{w_i}{N} \sum_{(s,a)} \log \Pi_\Phi(a \mid s)$$

- 12: **return** Π_Φ
- 13: **end function**
- 14: **function** COMPUTEREWARDS(F_{NE}, F_θ)
- 15: Compute scores $F_\theta(x_i)$ for all $x_i \in F_{NE}$
- 16: Estimate normalization statistics μ, σ
- 17: Assign weights:

$$w_i = \exp\left(\frac{F_\theta(x_i) - \mu}{\sigma\tau}\right)$$

- 18: **return** updated F_{NE}
- 19: **end function**
- 20: Initialize F_θ, Π_Φ
- 21: Initialize step size α , total steps T
- 22: $t \leftarrow 0$
- 23: **while** $t < T$ **do**
- 24: $F_\theta \leftarrow \text{TRAINDENSITY}(F_E, F_\theta)$
- 25: $F_{NE} \leftarrow \text{COMPUTEREWARDS}(F_{NE}, F_\theta)$
- 26: $\Pi_\Phi \leftarrow \text{TRAINBC}(F_{NE}, \Pi_\Phi)$
- 27: $t \leftarrow t + \alpha$
- 28: **end while**
- 29: **Final refinement:**
- 30: $F_{NE} \leftarrow \text{COMPUTEREWARDS}(F_{NE}, F_\theta)$
- 31: $\Pi_\Phi \leftarrow \text{TRAINBC}(F_{NE}, \Pi_\Phi)$

7.2 Potential Approaches for Parameter Tuning

As seen, a major issue with the existing distribution that was being estimated was its inability to distribute within a particular range and distinguish when where some trajectories are preferred over the other.

The temperature parameter was designed to control this spread, however, it was acting different for each environment, tuning it as an individual hyperparameter was difficult. One way to address this would be to define the exponential distribution to be spread out between 0 and 10.

The exponential distribution can then be defined as

$$P_\alpha(s) = \frac{\exp\left(\alpha \frac{R(s) - \mu_D}{\sigma_D}\right)}{\mathbb{Z}(\alpha)}$$

where $\mathbb{Z}(\alpha) = \sum_{s \in \mathcal{B}} \exp\left(\alpha \frac{R(s) - \mu_D}{\sigma_D}\right)$, $\bar{w}(s) = \alpha \frac{R(s) - \mu_D}{\sigma_D}$ and $\alpha = \frac{1}{\tau}$.

Let $q(s) = \frac{1}{|\mathcal{B}|}$. The objective to be minimized is

$$\mathcal{L}(\alpha) = \text{KL}(q(s) \parallel P_\alpha(s))$$

The expanded objective would be

$$\mathcal{L}(\alpha) = \mathbb{E}_{s \sim q} \left[\log q(s) - \alpha \frac{R(s) - \mu_D}{\sigma_D} + \log \mathbb{Z}(\alpha) \right]$$

Since $\log q(s)$ is a constant,

$$\mathcal{L}(\alpha) = -\alpha \mathbb{E}_{s \sim q} \left[\frac{R(s) - \mu_D}{\sigma_D} \right] + \log \mathbb{Z}(\alpha)$$

Since $\mathbb{E}_{s \sim q} \left[\frac{R(s) - \mu_D}{\sigma_D} \right]$ is 0, $\mathcal{L}(\alpha) = \log \mathbb{Z}(\alpha)$

Then

$$\nabla_\alpha \mathcal{L}(\alpha) = \mathbb{E}_{s \sim P_\alpha} \left[\frac{R(s) - \mu_D}{\sigma_D} \right]$$

The optimality condition then would be

$$\mathbb{E}_{s \sim P_\alpha} \left[\frac{R(s) - \mu_D}{\sigma_D} \right] = 0$$

This would be the convergence point for α . This could be iteratively included into the existing algorithm and the gradient can be calculated at each step to further push the scores up and down.

The three primary equations are thus:

$$P_\alpha(s) = \frac{\exp\left(\alpha \frac{R(s) - \mu_D}{\sigma_D}\right)}{\sum_{s' \in \mathcal{B}} \exp\left(\alpha \frac{R(s') - \mu_D}{\sigma_D}\right)}, \quad \alpha = \frac{1}{\tau}$$

$$\mathcal{L}(\alpha) = \text{KL}(q(s) \parallel P_\alpha(s)) = -\alpha \mathbb{E}_q \left[\frac{R(s) - \mu_D}{\sigma_D} \right] + \log \mathbb{Z}(\alpha)$$

$$\nabla_\alpha \mathcal{L}(\alpha) = \mathbb{E}_{P_\alpha} \left[\frac{R(s) - \mu_D}{\sigma_D} \right]$$

7.3 Baselines

As discussed before, the LobsDICE is consistently underperforming in all environments in v4, v5 and I am not able to recreate the results from the original paper.

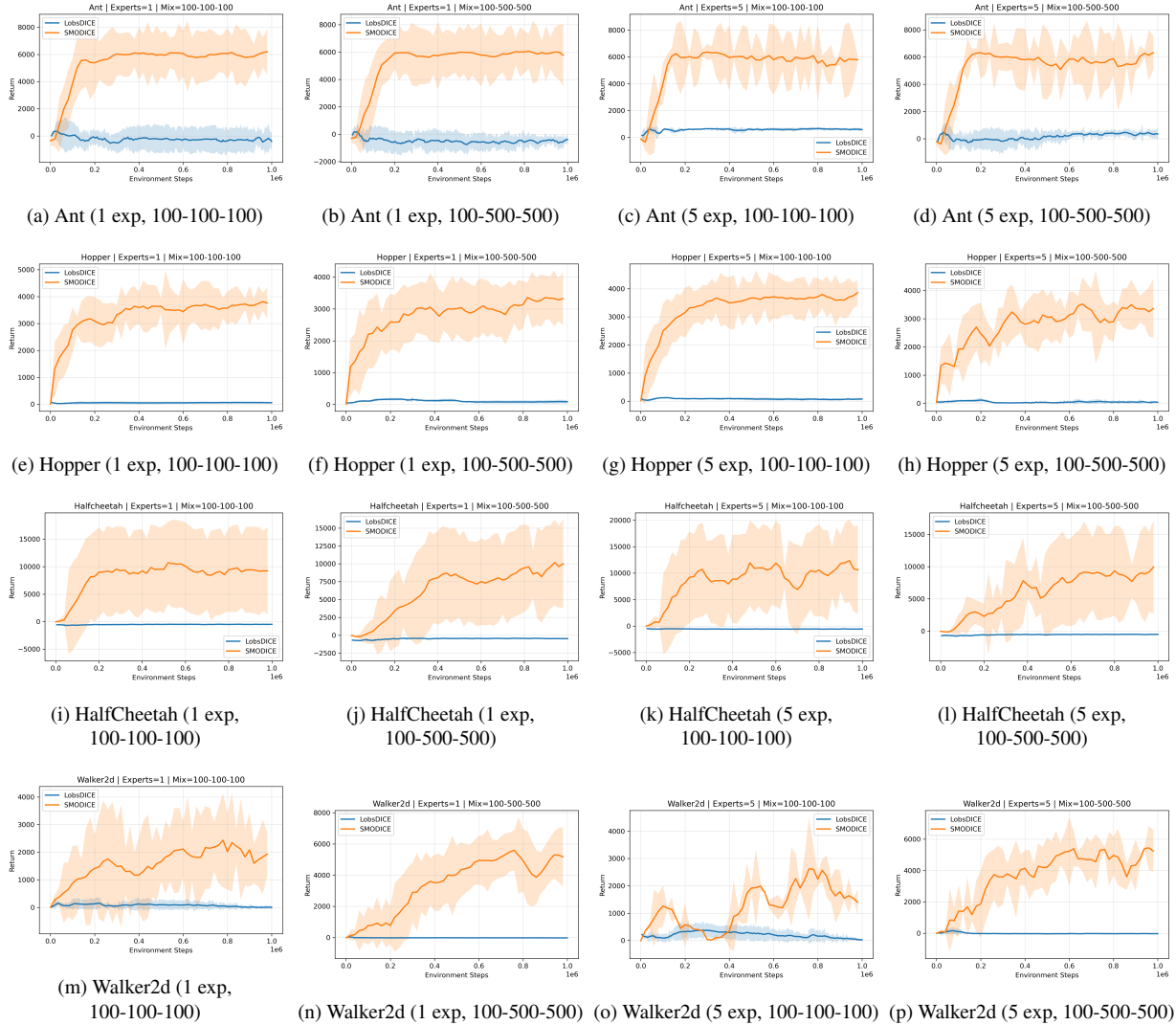


Figure 16: Comparison of LobsDICE and SMOODICE across four MuJoCo environments under varying expert dataset sizes and mixture distributions. Curves show mean return over training with shaded standard deviation across seeds.